数据结构 A 第 6 次作业

姓名: 王子川　　**学号:** PB25000055

1(4.17) 串的 Replace

用指针 i 指示当前扫描在串 S 中的位置, 使用 `Index` 函数定位串 S 中第一次匹配 T 的位置. 如果不存在, 直接跳出循环. 如果存在, i 入队, 跳过 t 个位置, 避免匹配重叠. 最终按照队列 Q 中记录下的匹配位置替换为 V.

```
Status replace(SString &S, SString T, SString V) {
    int s = S.length, t = T.length, v = V.length;
    if (t == 0)
        return ERROR;

    // Find all non-overlapping matches.
    SqQueue Q;
    InitQueue(Q);
    int i = 1, j = 1;
    while (i <= s-t+1) {
        i = Index(S, T, i);
        if (!i) break;
        EnQueue(Q, i);
        i += t;
    }

    // Check the resulting string length.
    int k = QueueLength(Q);
    int ls = s + k * (v-t);
    if (ls > MAXLEN)
        return ERROR;

    // Construct the new string.
    SString copyS, sub;
    copyS.length = 0;
    i = 1;
    while (!QueueEmpty(Q)) {
        DeQueue(Q, j);
        SubString(sub, S, i, j-i);
        Concat(copyS, copyS, sub);
        Concat(copyS, copyS, V);
        i = j+t;
    }

    // Append the remaining suffix.
    if (i <= s) {
        SubString(sub, S, i, s-i+1);
        Concat(copyS, copyS, sub);
    }

    // Update the original string.
    StrCopy(S, copyS);
    return OK;
}
```

设原串长度为 s, 模式串长度为 t, 替换串长度为 v, 匹配次数为 k. 查找阶段: 如果 Index 使用 KMP 算法, 最坏时间复杂度为 $O(s+t)$. 构造阶段: 需要 $O(k(s+kv))$ 时间. 综上, 算法的时间复杂度为 $O(s+t+k(s+kv))$.

2(4.21) 串的 StrAssign, StrCopy, StrCompare, StrLength, Concat, SubString

```cpp
struct LNode {
    char ch;
    LNode *next;
};

struct LString {
    LNode head = {'\0', nullptr};
};

// Clear all character nodes.
void ClearString(LString &S) {
    LNode *p = S.head.next;
    while (p) {
        LNode *q = p;
        p = p->next;
        delete q;
    }
    S.head.next = nullptr;
}

// Append a character to a linked list.
void Append(LNode *&tail, char c) {
    tail->next = new LNode{c, nullptr};
    tail = tail->next;
}

// Replace the old character nodes.
void ReplaceNodes(LString &T, LNode *first) {
    ClearString(T);
    T.head.next = first;
}

// (1) StrAssign
Status StrAssign(LString &T, const char *chars) {
    if (!chars) return ERROR;

    LNode head{'\0', nullptr};
    LNode *tail = &head;
    for (int i = 0; chars[i] != '\0'; i++)
        Append(tail, chars[i]);

    ReplaceNodes(T, head.next);
    return OK;
}

// (2) StrCopy
Status StrCopy(LString &T, const LString &S) {
    LNode head{'\0', nullptr};
    LNode *tail = &head;
    for (LNode *p = S.head.next; p; p = p->next)
```

```
        Append(tail, p->ch);

    ReplaceNodes(T, head.next);
    return OK;
}

// (3) StrCompare
int StrCompare(const LString &S, const LString &T) {
    LNode *p = S.head.next, *q = T.head.next;
    while (p && q) {
        if (p->ch != q->ch)
            return (unsigned char)p->ch - (unsigned char)q->ch;
        p = p->next;
        q = q->next;
    }

    if (p) return 1;
    if (q) return -1;
    return 0;
}

// (4) StrLength
int StrLength(const LString &S) {
    int len = 0;
    for (LNode *p = S.head.next; p; p = p->next)
        len++;
    return len;
}

// (5) Concat
Status Concat(LString &T, const LString &S1, const LString &S2) {
    LNode head{'\0', nullptr};
    LNode *tail = &head;

    for (LNode *p = S1.head.next; p; p = p->next)
        Append(tail, p->ch);
    for (LNode *p = S2.head.next; p; p = p->next)
        Append(tail, p->ch);

    ReplaceNodes(T, head.next);
    return OK;
}

// (6) SubString
Status SubString(LString &Sub, const LString &S, int pos, int len) {
    int n = StrLength(S);
    if (pos < 1 || pos > n || len < 0 || len > n-pos+1)
        return ERROR;

    LNode *p = S.head.next;
    for (int i = 1; i < pos; i++)
        p = p->next;

    LNode head{'\0', nullptr};
    LNode *tail = &head;
    for (int i = 0; i < len; i++) {
        Append(tail, p->ch);
```

```
        p = p->next;
    }

    ReplaceNodes(Sub, head.next);
    return OK;
}
```

上面的操作的时间复杂度都是线性在字符串长度的.

3(4.23) 判定链串的对称性

题目给定链串, 要判定其回文性. 思路同上次作业 3.31.

```
Status Palindrome(LString T) {
    Stack S;
    InitStack(S);
    int len = T.curlen, cnt = 0;

    // Push all characters onto the stack.
    for (Chunk *p = T.head; p && cnt < len; p = p->next) {
        for (int i = 0; i < CHUNKSIZE && cnt < len; i++, cnt++)
            Push(S, p->ch[i]);
    }

    // Compare characters in reverse order.
    cnt = 0;
    for (Chunk *p = T.head; p && cnt < len; p = p->next) {
        for (int i = 0; i < CHUNKSIZE && cnt < len; i++, cnt++) {
            char c;
            Pop(S, c);
            if (c != p->ch[i]) {
                DestroyStack(S);
                return FALSE;
            }
        }
    }

    DestroyStack(S);
    return TRUE;
}
```

设字符串长度为 n. 每个字符至多出入栈各一次, 因此时空复杂度均为 $O(n)$.